
IBM Data Science Professional Certificate

Capstone Project

Skills Network

Student: Leonardo Vittar
Year: 2026



SpaceX

Falcon 9 Launches Analytics

Summary

Executive Summary.....	7
Introduction.....	8
Understanding the competitor.....	8
Some SpaceX Facts.....	9
Reusable launch system program.....	10
Methodology.....	11
Data Collection and Wrangling.....	13
Data Collection From SpaceX REST API.....	15
Data Collection From SpaceX Wikipage.....	17
Data Wrangling From SpaceX REST API.....	19
Data Wrangling From SpaceX Wikipage.....	21
Exploratory Data Analysis (EDA).....	21
Percentage of the missing values in each attribute,.....	21
Numerical and categorical columns,.....	22
Number of Launches per Site,.....	22
Orbit Related Definitions (Common orbit types).....	23
Number of Launches per Orbit,.....	24
Landings Outcomes,.....	24
Missions Outcomes Names Meanings,.....	25
Bad Outcomes,.....	25
Success Rate,.....	25
Exploratory Data Analysis SQL (EDA).....	26
Statistical analysis and data visualization.....	35
Payload Mass vs. Flight Number.....	35
Launch Site vs. Flight Number.....	36
Launch Site vs. Payload Mass.....	37
Success Rate vs. Orbit.....	38
Orbit vs. Flight Number.....	39

Orbit vs. Payload Mass.....	40
Success Rate vs. Year.....	41
Folium Launch Sites Maps.....	42
Plotly Dash Interactive Charts.....	46
Adding a Launch Site Drop-down Input Component,.....	46
Success-pie-chart site dropdown callback function,.....	47
Adding a Range Slider to Select Payload,.....	48
Success-payload scatter-chart callback function to render the scatter plot,.....	48
Different sites selections pie charts,.....	49
Different slider settings scatter charts,.....	50
The site with largest successful launches & the highest launch success rate is KSC LC-39A as you can see in the chart below,....	51
The payload range with the highest launch success rate is 2940Kg - 6761Kg.....	52
corresponding to KSC LC-39A, as you can see in the below figures,.....	52
The site with the lowest launch success rate is CCAFS SLC-40,.....	53
Predictive models (Building, Evaluating, Refining).....	56
Logistic Regression Model.....	57
Support Vectors Machine Model (SVM).....	59
Tree Classifier Model.....	61
K_Nearest_Neighbors Classiffier (KNN).....	63
Conclusion.....	65
Repository Links:.....	67



Executive Summary

IBM-Skills Network Capstone Project initial premise,

In this capstone project, we will predict if the Falcon 9 first stage will land successfully.

SpaceX advertises Falcon 9 rocket launches on its website with a cost of 62 million dollars; other providers cost upward of 165 million dollars each, much of the savings is because SpaceX can reuse the first stage. Therefore if we can determine if the first stage will land, we can determine the cost of a launch. This information can be used if an alternate company wants to bid against SpaceX for a rocket launch.

“Most unsuccessful landings are planned.”

“Space X performs a controlled landing in the oceans. ”

Introduction

Understanding the competitor

The goal of this Data Science Project, is to apport valuable information on SpaceX Falcon 9 launches to a new spacial company that pretend to be its competitor.

The Data to be analyzed are that exposed in the SpaceX API, and the ones from a set of tables published in Wikipedia. Both resources are provided by Skills Networks.

We will analyze the data looking for deciphering the criteria that define the SpaceX desitions about the service type that it will provide to its customers and so its costs and services price.

Some SpaceX Facts

Trade name: SpaceX
Company type: Private
Industry: Space Telecommunications Artificial intelligence
Founded: March 14, 2002 in El Segundo, California, U.S.
Founder: Elon Musk
Headquarters: SpaceX Starbase, Starbase, Texas.

***SpaceX** has significantly transformed the global aerospace industry by reducing launch costs through its **reusable rocket technology**, making space more accessible and affordable.*

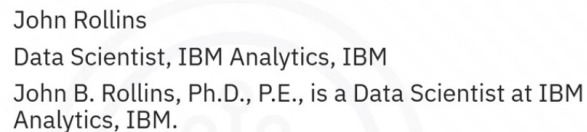
*This has led to increased competition, prompting both established companies and new entrants to innovate and adapt, while **SpaceX** has captured a dominant market share in commercial launches and satellite services.*

Reusable launch system program

Some chronology

- ◎ The program was announced in 2011.
- ◎ SpaceX first achieved a successful landing and recovery of a first stage in December 2015.
- ◎ The first re-flight of a landed first stage occurred in March 2017 with the second occurring in June 2017, that one only five months after the maiden flight of the booster.
- ◎ The third attempt occurred in October 2017 with the SES-11/EchoStar-105 mission.
- ◎ Reflights of refurbished first stages then became routine.
- ◎ In May 2021, B1051 became the first booster to launch ten missions.

In this Data Science project our methodology will be a condensed version of that outlined by the IBM Data Scientist John Rollins Ph.D.,



```
graph TD; BU[Business understanding] --> AP[Analytic approach]; AP --> DR[Data requirements]; DR <--> DC[Data collection]; DC <--> DU[Data understanding]; DU --> DP[Data preparation]; DP --> M[Modeling]; M --> E[Evaluation]; E --> D[Deployment]; D --> F[Feedback]; F --> M; M --> DR; M --> DP;
```

11 / 67



For this Capstone Project we will specifically define the following stages,

- ✓ Data collection from various sources,
- ✓ Data Quality improving by performing data wrangling,
- ✓ Exploration of the processed data, query the data and gather insights.
- ✓ Some statistical analysis and data visualization, to see how variables are related to each other.
- ✓ Drill down into finer levels of detail by splitting the data into groups defined by categorical variables or factors in your data
- ✓ Predictive models building, evaluating, and refining,

Data Collection and Wrangling

In this stage we collect SpaceX launch data making a get request to the SpaceX API. We also do some basic data wrangling and formating.

- Request to the SpaceX API
- Clean the requested data

SpaceX launch data that is gathered from the SpaceX REST API, giving us data about launches, including,

- rocket used,
- payload delivered,
- launch specifications,
- landing specifications,
- landing outcome,

The SpaceX REST API endpoints, or URL, starts with api.spacexdata.com/v4/.

Even being "<https://api.spacexdata.com/v4/launches/past>" the original URL, for a matter of consistence, to request and parse the SpaceX launch data using the GET request we will use the below link provided for IBM-SkillsNetwork,

`static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/API_call_spacex_api.json'`

Another popular Falcon 9 Launch data source are related Wiki pages. Using the Python BeautifulSoup package we web scrape some HTML tables that contain valuable Falcon 9 launch records. Then we parse the data from those tables and convert them into a Pandas data frame for further visualization and analysis.

Data Collection From SpaceX REST API

Building a dataframe from the normalized json,

```
# Use json_normalize meethod to convert the json result into a dataframe
data = pd.json_normalize(response.json())
```

Table from the normalized json,

```
# Get the head of the dataframe
data.head()
```

	static_fire_date_utc	static_fire_date_unix	net	window	rocket	success	failures	details	crew	ships	capsules
0	2006-03-17T00:00:00.000Z	1.142554e+09	False	0.0	5e9d0d95eda69955f709d1eb	False	[{'time': 33, 'altitude': None, 'reason': 'merlin engine failure'}]	Engine failure at 33 seconds and loss of vehicle	[]	[]	[5eb0e4b5b6c3bb00]
1	None	NaN	False	0.0	5e9d0d95eda69955f709d1eb	False	[{'time': 301, 'altitude': 289, 'reason': 'harmonic oscillation leading to premature engine shutdown'}]	Successful first stage burn and transition to second stage, maximum altitude 289 km, Premature engine shutdown at T+7 min 30 s, Failed to reach orbit, Failed to recover first stage	[]	[]	[5eb0e4b5b6c3bb00]

Building a dataframe from selected columns of the table,

```
# Show the head of the dataframe
df.head()
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial
0	1	2006-03-24	Falcon 1	20.0	LEO	Kwajalein Atoll	None None	1	False	False	False	None	NaN	0	Merlin1A 1
1	2	2007-03-21	Falcon 1	NaN	LEO	Kwajalein Atoll	None None	1	False	False	False	None	NaN	0	Merlin2A 1
2	4	2008-09-28	Falcon 1	165.0	LEO	Kwajalein Atoll	None None	1	False	False	False	None	NaN	0	Merlin2C 1
3	5	2009-07-13	Falcon 1	200.0	LEO	Kwajalein Atoll	None None	1	False	False	False	None	NaN	0	Merlin3C 1
4	6	2010-06-04	Falcon 9	NaN	LEO	CCSFS SLC 40	None None	1	False	False	False	None	1.0	0	B0003

Filtered dataframe with only Falcon 9 records ('FlightNumber' index reseted),

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount
0	1	2010-06-04	Falcon 9	NaN	LEO	CCSFS SLC 40	None None	1	False	False	False	None	1.0	0
1	2	2012-05-22	Falcon 9	525.0	LEO	CCSFS SLC 40	None None	1	False	False	False	None	1.0	0
2	3	2013-03-01	Falcon 9	677.0	ISS	CCSFS SLC 40	None None	1	False	False	False	None	1.0	0
3	4	2013-09-29	Falcon 9	500.0	PO	VAFB SLC 4E	False Ocean	1	False	False	False	None	1.0	0
4	5	2013-12-03	Falcon 9	3170.0	GTO	CCSFS SLC 40	None None	1	False	False	False	None	1.0	0

Data Collection From SpaceX Wikipage

We will performing web scraping using BeautifulSoup to collect Falcon 9 historical launch records from a Wikipedia page.

Even being “https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches” the original URL, for a matter of consistance we will scrape the data from a snapshot of the “List of Falcon 9 and Falcon Heavy launches” Wikipage updated on 9th June 2021 from the below url provided for IBM-SkillsNetwork,

static_url = https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922

We will,

- Use HTTP GET method to request the Falcon9 Launch HTML page,
- Create a BeautifulSoup object from the HTML response,
- Verify if the BeautifulSoup object printing the page title,
- Collect all relevant column names from the HTML table header,

- Create an empty dictionary with keys from the extracted column names,
- Convert the dictionary into a Pandas dataframe,

Column names from the HTML table header

```
print(column_names)
```

```
['Flight No.', 'Date and time ( )', '', 'Launch site', 'Payload', 'Payload mass', 'Orbit', 'Customer', 'Launch outcome', '', None, None, None, None, None, None, None]
```

Pandas Dataframe

	Flight No.	Unnamed: 1	Launch site	Payload	Payload mass	Orbit	Customer	Launch outcome	Unnamed: 8	Version Booster	Booster landing	Date	Time
0	1	NaN	CCAFS	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success\n	NaN	F9 v1.1	Failure	4 June 2010	18:45
1	2	NaN	CCAFS	Dragon	0	LEO	NASA	Success	NaN	F9 v1.1	Failure	8 December 2010	15:43
2	3	NaN	CCAFS	Dragon	525 kg	LEO	NASA	Success	NaN	F9 v1.1	No attempt\n	22 May 2012	07:44
3	4	NaN	CCAFS	SpaceX CRS-1	4,700 kg	LEO	NASA	Success\n	NaN	F9 v1.1	No attempt	8 October 2012	00:35
4	5	NaN	CCAFS	SpaceX CRS-2	4,877 kg	LEO	NASA	Success\n	NaN	F9 v1.1	No attempt\n	1 March 2013	15:10

Data Wrangling From SpaceX REST API

In the figure below you can see two columns with missing values, but the only important ones are the five missing values of the 'PayloadMass' column.

```
data_falcon9.isnull().sum()
```

FlightNumber	0
Date	0
BoosterVersion	0
PayloadMass	5
Orbit	0
LaunchSite	0
Outcome	0
Flights	0
GridFins	0
Reused	0
Legs	0
LandingPad	26
Block	0
ReusedCount	0
Serial	0
Longitude	0
Latitude	0
dtype:	int64

Calculating the mean payload mass,

```
pl_mean = data_falcon9['PayloadMass'].mean()
data_falcon9['PayloadMass'] = data_falcon9['PayloadMass'].fillna(pl_mean)
data_falcon9.isnull().sum()
```

filling the Missing Values with that Mean,

```
FlightNumber      0
Date              0
BoosterVersion    0
PayloadMass       0
Orbit             0
LaunchSite        0
Outcome           0
Flights           0
GridFins          0
Reused            0
Legs              0
LandingPad        26
Block             0
ReusedCount       0
Serial            0
Longitude         0
Latitude          0
dtype: int64
```


Data Wrangling From SpaceX Wikipage

We will perform some Exploratory Data Analysis to find some patterns in the data and determine what would be the label for training supervised models.

Exploratory Data Analysis (EDA)

Percentage of the missing values in each attribute,

```
df.isnull().sum()/len(df)*100
```

FlightNumber	0.000000
Date	0.000000
BoosterVersion	0.000000
PayloadMass	0.000000
Orbit	0.000000
LaunchSite	0.000000
Outcome	0.000000
Flights	0.000000
GridFins	0.000000
Reused	0.000000
Legs	0.000000
LandingPad	28.888889
Block	0.000000
ReusedCount	0.000000
Serial	0.000000
Longitude	0.000000
Latitude	0.000000
dtype:	float64

Numerical and categorical columns,

```
df.dtypes
```

```
FlightNumber    int64  
Date            object  
BoosterVersion  object  
PayloadMass     float64  
Orbit           object  
LaunchSite      object  
Outcome         object  
Flights         int64  
GridFins        bool  
Reused          bool  
Legs            bool  
LandingPad      object  
Block           float64  
ReusedCount     int64  
Serial          object  
Longitude       float64  
Latitude        float64  
dtype: object
```

Number of Launches per Site,

```
df['LaunchSite'].value_counts()
```

```
LaunchSite  
CCAFS SLC 40    55  
KSC LC 39A      22  
VAFB SLC 4E     13  
Name: count, dtype: int64
```


Orbit Related Definitions (Common orbit types)

LEO (Low Earth orbit)	- most of the manmade objects in outer space are in LEO, - earth-centred orbit, - altitude of 2,000 km (1,200 mi) or less (approx. one-third of the Earth radius), - at least 11.25 periods per day (an orbital period of 128 minutes or less), - eccentricity less than 0.25.
VLEO (Very Low Earth Orbit)	- mean altitude below 450 km. Operating in these orbits can provide a number of benefits to Earth observation spacecraft as the spacecraft operates closer to the observation[2].
GTO (Geostationary Transfer Orbit)	- elliptical Earth orbit used to transfer satellites from low Earth orbit (LEO) to geostationary orbit (GEO), - perigee (closest point to Earth) much lower than GEO altitude, - apogee (farthest point) reaches approximately 22,236 miles (35,786 kilometers) above Earth's equator the altitude of a geostationary orbit, - satellites use onboard propulsion to circularize their orbit at GEO altitude, where they can provide services such as weather monitoring, communications, and surveillance.
SSO or SO (Sun Synchronous Orbit or heliosynchronous)	- nearly polar orbit around a planet, - satellite passes over any given point of the planet's surface at the same local mean solar time.
ES-L1 (Lagrange point between the sun and the earth)	- At the Lagrange points the gravitational forces of the two large bodies cancel out in such a way that a small object placed in orbit there is in equilibrium relative to the center of mass of the large bodies.
HEO (Highly Elliptical Orbit)	- elliptic orbit with high eccentricity, usually referring to one around Earth.
ISS (International Space Station)	- low Earth orbit - multinational collaborative project (NASA (United States), Roscosmos (Russia), JAXA (Japan), ESA (Europe), and CSA (Canada))
MEO (Geocentric orbits AKA intermediate circular orbit)	- altitude 2,000 km (1,200 mi) to 35,786 kilometers (22,236 mi), just below geosynchronous orbit, - most commonly at 20,200 kilometers (12,600 mi), or 20,650 kilometers (12,830 mi), - orbital period of 12 hours
HEO (Geocentric orbits)	- 35,786 km or 22,236 mi (above the altitude of geosynchronous orbit).
GEO (Circular Geosynchronous Orbit):	- 35,786 kilometres (22,236 miles) above Earth's equator and following the direction of Earth's rotation.
PO (Polar Orbits)	- It is one type of satellites in which a satellite passes above or nearly above both poles of the body being orbited (usually a planet such as the Earth).

Number of Launches per Orbit,

```
df['Orbit'].value_counts()

Orbit
GT0      27
ISS      21
VLE0     14
P0        9
LE0        7
SS0        5
ME0        3
HE0        1
ES-L1     1
S0         1
GE0        1
Name: count, dtype: int64
```

Landings Outcomes,

```
landing_outcomes = df['Outcome'].value_counts()
landing_outcomes

Outcome
True ASDS      41
None None      19
True RTLS      14
False ASDS       6
True Ocean       5
False Ocean      2
None ASDS        2
False RTLS        1
Name: count, dtype: int64
```

Missions Outcomes Names Meanings,

True Ocean	Landed to a specific region of the ocean
False Ocean	Unsuccessfully landed to a specific region of the ocean
True RTLS	Successfully landed to a ground pad
False RTLS	Unsuccessfully landed to a ground pad
True ASDS	Successfully landed to a drone ship
False ASDS	Unsuccessfully landed to a drone ship
None ASDS	Failure to land
None None	Failure to land

Bad Outcomes,

```
bad_outcomes=set(landing_outcomes.keys()[[1,3,5,6,7]])  
bad_outcomes  
  
{'False ASDS', 'False Ocean', 'False RTLS', 'None ASDS', 'None None'}
```

Success Rate,

```
df["Class"].mean()  
  
np.float64(0.6666666666666666)
```

Exploratory Data Analysis SQL (EDA)

- Download the datasets from the link below,
https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/labs/module_2/data/Spacex.csv,
- Load the spacex dataset,
- Load the SQL extension and establish a connection with the database,

```
# !pip install ipython-sql
# !pip install ipython-sql prettytable

%load_ext sql

import csv, sqlite3
import prettytable
prettytable.DEFAULT = 'DEFAULT'

con = sqlite3.connect("my_data1.db")
cur = con.cursor()

# !pip install -q pandas

%sql sqlite:///my_data1.db
```


Exploring Tables,

```
%sql SELECT name as SQLite_Tables FROM sqlite_schema WHERE type='table' AND name NOT LIKE 'sqlite_%';
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
SQLite_Tables
```

```
SPACEXTABLE
```

```
SPACEXTBL
```

Visualizing old SPACEXTABLE,

```
%sql SELECT * FROM SPACEXTABLE LIMIT 10;
```

```
* sqlite:///my_data1.db
```

```
Done.
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Erasing old SPACEXTABLE table,

```
#DROP THE TABLE IF EXISTS
%sql DROP TABLE IF EXISTS SPACEXTABLE;

* sqlite:///my_data1.db
Done.
[]
```

Creating a new SPACEXTABLE Table from SPACEXTBL,

```
%sql create table SPACEXTABLE as select * from SPACEXTBL where Date is not null

* sqlite:///my_data1.db
Done.
[]
```

Names of the unique launch sites in the space mission,

```
# %sql SELECT DISTINCT "Launch_Site" FROM SPACEXTABLE
%sql SELECT DISTINCT "Launch_Site" as "Launch_Site_Names" FROM SPACEXTABLE

* sqlite:///my_data1.db
Done.

Launch_Site_Names
-----
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40
```


5 records where launch sites begin with the string 'CCA',

```
%sql SELECT * FROM SPACEXTABLE WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5;
```

```
* sqlite:///my_data1.db  
Done.
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total payload mass carried by boosters launched by NASA (CRS),

```
%sql SELECT SUM (PAYLOAD_MASS_KG_) FROM SPACEXTABLE WHERE "Customer" LIKE '%(CRS)';
```

```
* sqlite:///my_data1.db  
Done.
```

SUM (PAYLOAD_MASS_KG_)
45596

Average payload mass carried by booster version F9 v1.1,

```
%sql SELECT AVG (PAYLOAD_MASS_KG_) FROM SPACEXTABLE WHERE "Booster_Version" LIKE '%F9 v1.1%';  
* sqlite:///my_data1.db  
Done.  
AVG (PAYLOAD_MASS_KG_)  
-----  
2534.6666666666665
```

Date when the first succesful landing outcome in ground pad was acheived,

```
%%sql  
SELECT MIN ("Date") as "First_Success_Date", "Mission_Outcome", "Landing_Outcome"  
FROM SPACEXTABLE  
WHERE Mission_Outcome LIKE '%Succes%'  
AND Landing_Outcome LIKE '%(ground pad)%';  
  
* sqlite:///my_data1.db  
Done.  
First_Success_Date  Mission_Outcome  Landing_Outcome  
-----  
2015-12-22          Success      Success (ground pad)
```

Names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000,

```
%%sql
SELECT "Booster_Version", PAYLOAD_MASS_KG_ as "Payload_Mass_(Kg)", "Landing_Outcome"
FROM SPACEXTABLE WHERE PAYLOAD_MASS_KG_ > 4000 AND "Landing_Outcome" LIKE '%Success (drone ship)';
```

```
* sqlite:///my_data1.db
Done.
```

Booster_Version	Payload_Mass_(Kg)	Landing_Outcome
F9 FT B1022	4696	Success (drone ship)
F9 FT B1026	4600	Success (drone ship)
F9 FT B1029.1	9600	Success (drone ship)
F9 FT B1021.2	5300	Success (drone ship)
F9 FT B1036.1	9600	Success (drone ship)
F9 B4 B1041.1	9600	Success (drone ship)
F9 FT B1031.2	5200	Success (drone ship)

Total number of successful and failure mission outcomes,

```
%%sql SELECT COUNT ("Mission_Outcome") FROM SPACEXTABLE WHERE ("Mission_Outcome" LIKE '%Success%') OR ("Mission_Outcome" LIKE '%Failure%');
```

```
* sqlite:///my_data1.db
Done.
```

COUNT ("Mission_Outcome")
101

Booster_versions that have carried the maximum payload mass,

```
%sql SELECT "Booster_Version" as "Max Payload Mass Boosters",  
PAYLOAD_MASS_KG_ as "Payload Mass"  
FROM SPACEXTABLE WHERE PAYLOAD_MASS_KG_ == (SELECT MAX (PAYLOAD_MASS_KG_) FROM SPACEXTABLE);
```

```
* sqlite:///my_data1.db
```

Done.

Max Payload Mass Boosters	Payload Mass
F9 B5 B1048.4	15600
F9 B5 B1049.4	15600
F9 B5 B1051.3	15600
F9 B5 B1056.4	15600
F9 B5 B1048.5	15600
F9 B5 B1051.4	15600
F9 B5 B1049.5	15600
F9 B5 B1060.2	15600
F9 B5 B1058.3	15600
F9 B5 B1051.6	15600
F9 B5 B1060.3	15600
F9 B5 B1049.7	15600

Records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

```
%%sql
SELECT substr(Date,0,5) as Year,
CASE substr(Date,6,2)
when '01' then 'January'
when '02' then 'February'
when '03' then 'March'
when '04' then 'April'
when '05' then 'May'
when '06' then 'June'
when '07' then 'July'
when '08' then 'August'
when '09' then 'September'
when '10' then 'October'
when '11' then 'November'
when '12' then 'December'
else '' end
as "Month" ,
"Landing_Outcome" as "Failure_Landing_Outcomes",
"Booster_Version",
"Launch_Site"
FROM SPACEXTABLE
WHERE "Landing_Outcome" LIKE 'Failure (drone ship)%'
AND substr(Date,0,5)='2015';
```

* sqlite:///my_data1.db

Done.

Year	Month	Failure_Landing_Outcomes	Booster_Version	Launch_Site
2015	January	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
2015	April	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order,

```
%%sql
SELECT COUNT ("Landing_Outcome") as "Count", "Landing_Outcome"
FROM SPACEXTABLE WHERE "Date"
BETWEEN '2010-06-04' and '2017-03-20'
GROUP BY "Landing_Outcome" ORDER BY "Count";
```

```
* sqlite:///my_data1.db
```

```
Done.
```

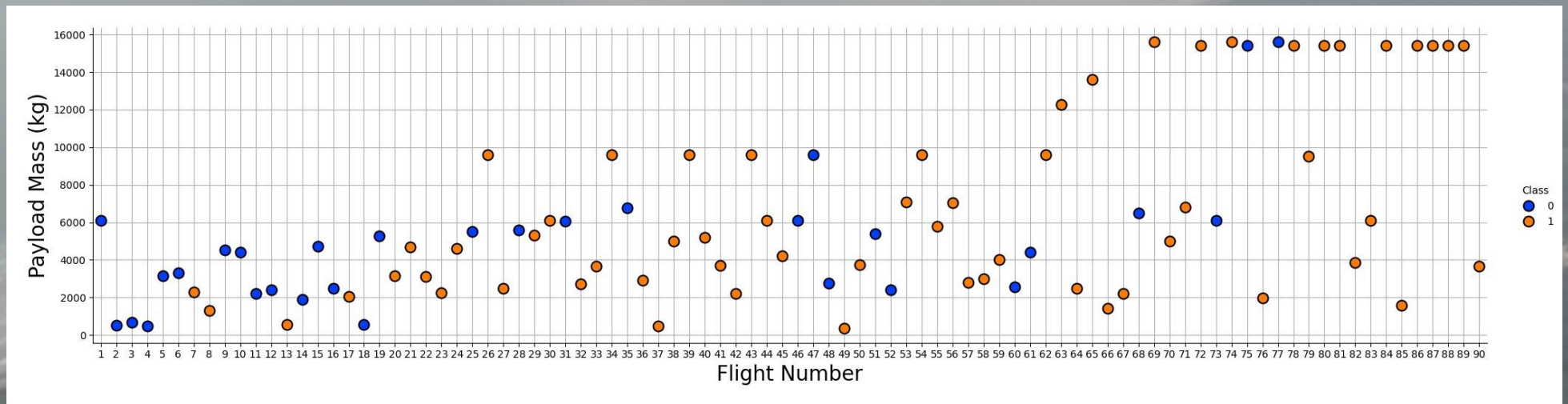
Count	Landing_Outcome
1	Precluded (drone ship)
2	Failure (parachute)
2	Uncontrolled (ocean)
3	Controlled (ocean)
3	Success (ground pad)
5	Failure (drone ship)
5	Success (drone ship)
10	No attempt

Statistical analysis and data visualization

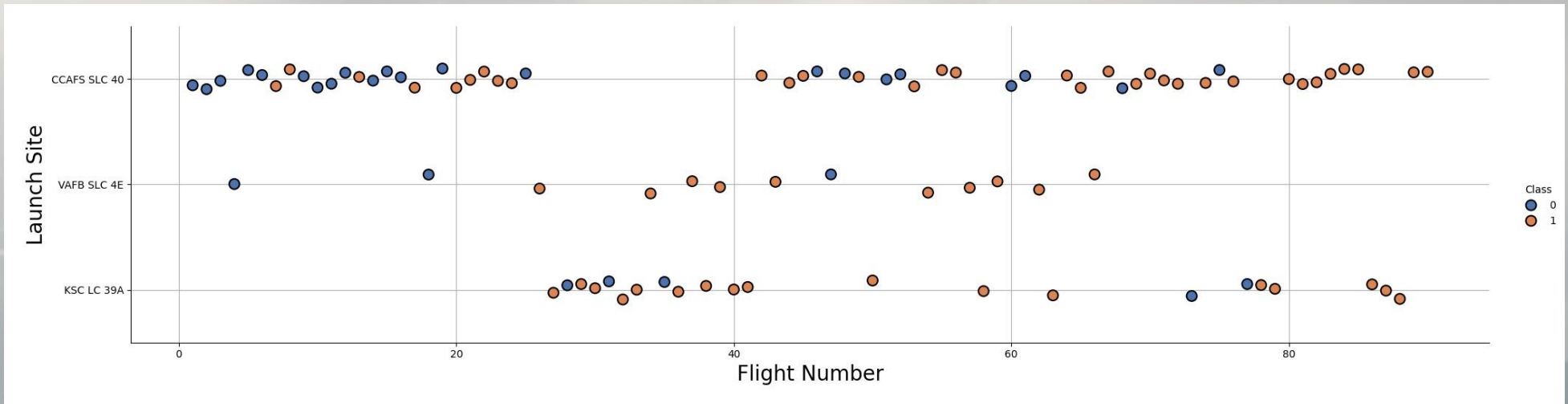
In the following plots, “orange dots indicate rocket was landing” and “blue dots that was not”.

Payload Mass vs. Flight Number

In the figure below we can see that as the flight number increases the first stage is more likely to land successfully. The payload mass is also important it seems the more massive the payload, the less likely the first stage will return.



Launch Site vs. Flight Number



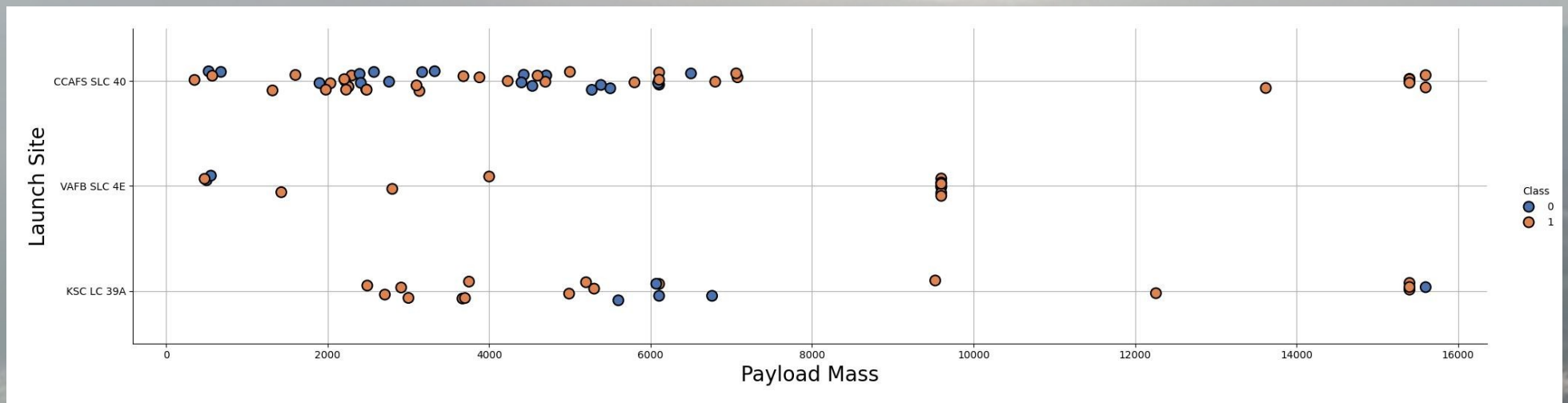
In the figure above we can see that the majority of the missions were launched from CCAFS SLC-40 with the following succes rate for each launch site,

- **CCAPS LC-40 60%** from a total 55 launches,
- **VABF SLC-4E about 77%** from a total of 13 launches,
- **KSC LC-39A about 77%** over 22 launches,

Launch Site vs. Payload Mass

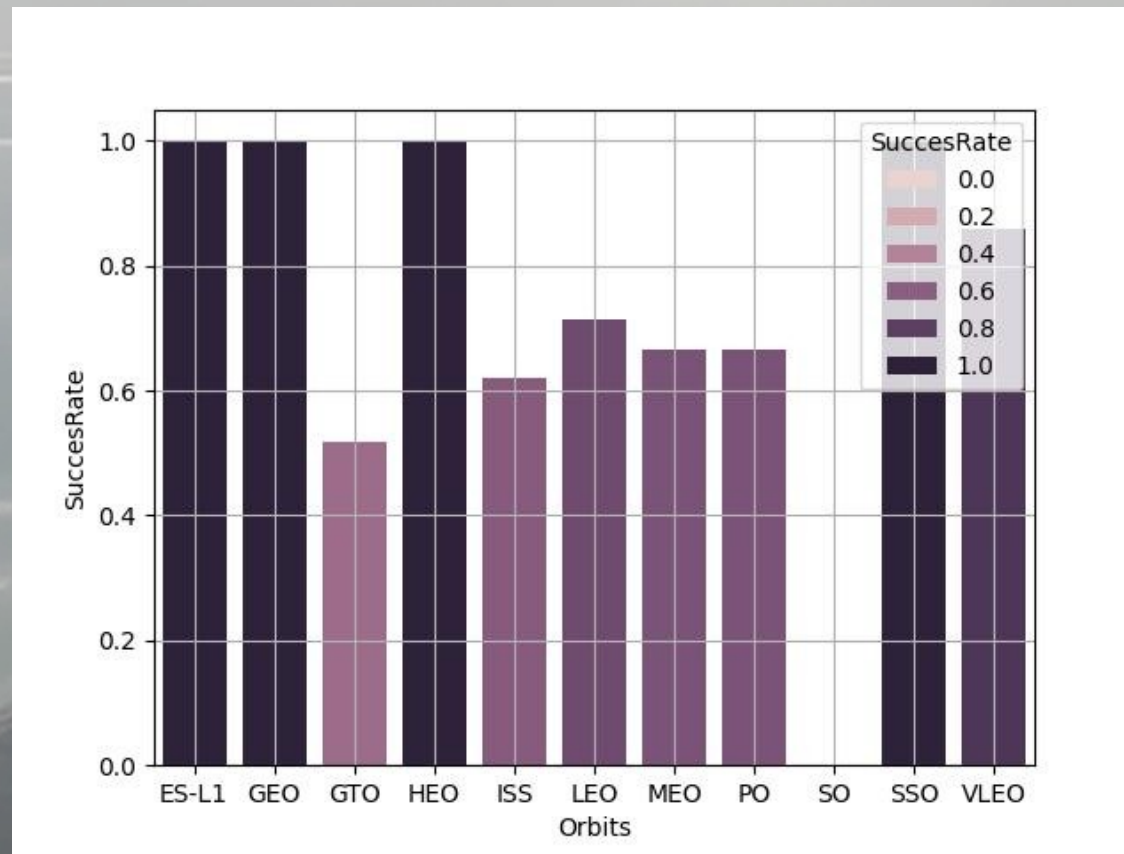
In the following plot we can essentially see,

- **CCAFS SLC-40** with **Payload Mass** ranging between about **500Kg to 7000Kg** with outliers of about 15000Kg,
- **KSC LC-39A** with **Payload Mass** ranging between about **1500Kg to 7000Kg** with outliers of about 15000Kg,



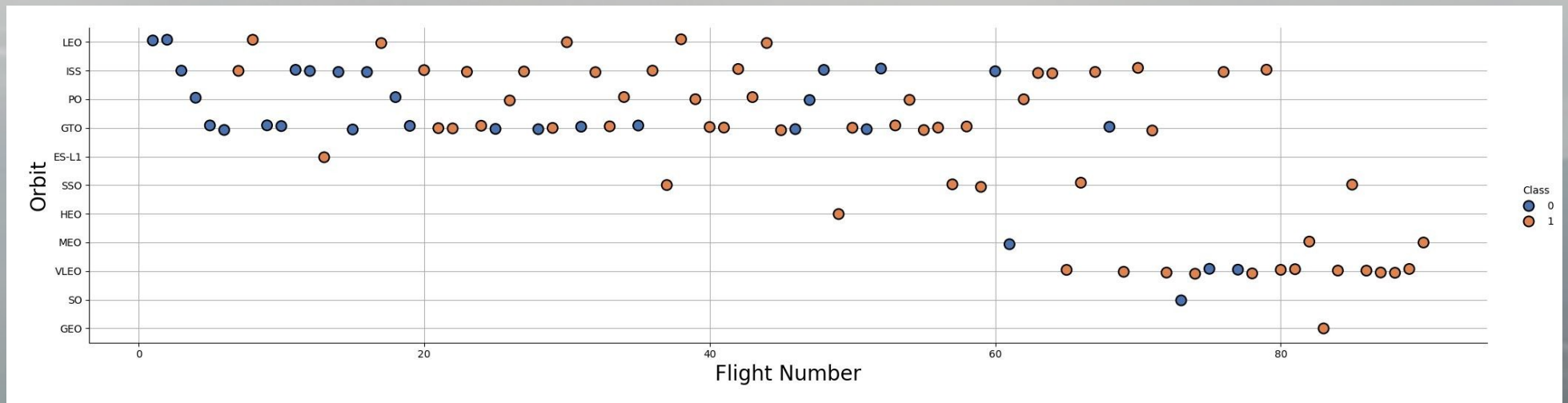
Succes Rate vs. Orbit

In the figure below we can see that **ES-L1**, **GEO**, **HEO** and **SSO** orbits have the maximum succes rate that is “1”, and **GTO**, **ISS**, **LEO**, **MEO**, and **PO** with succes rate ranging between 0,5 to 0,7.



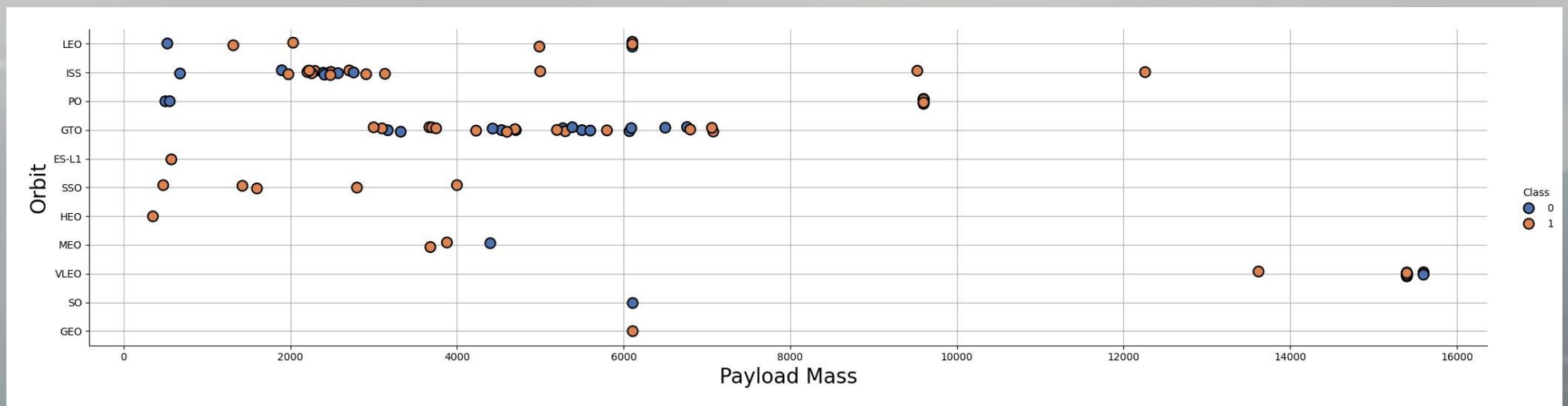
Orbit vs. Flight Number

In the figure below we can see that in the LEO orbit the Success appears related to the number of flights; on the other hand, there seems to be no relationship between flight number when in GTO orbit.

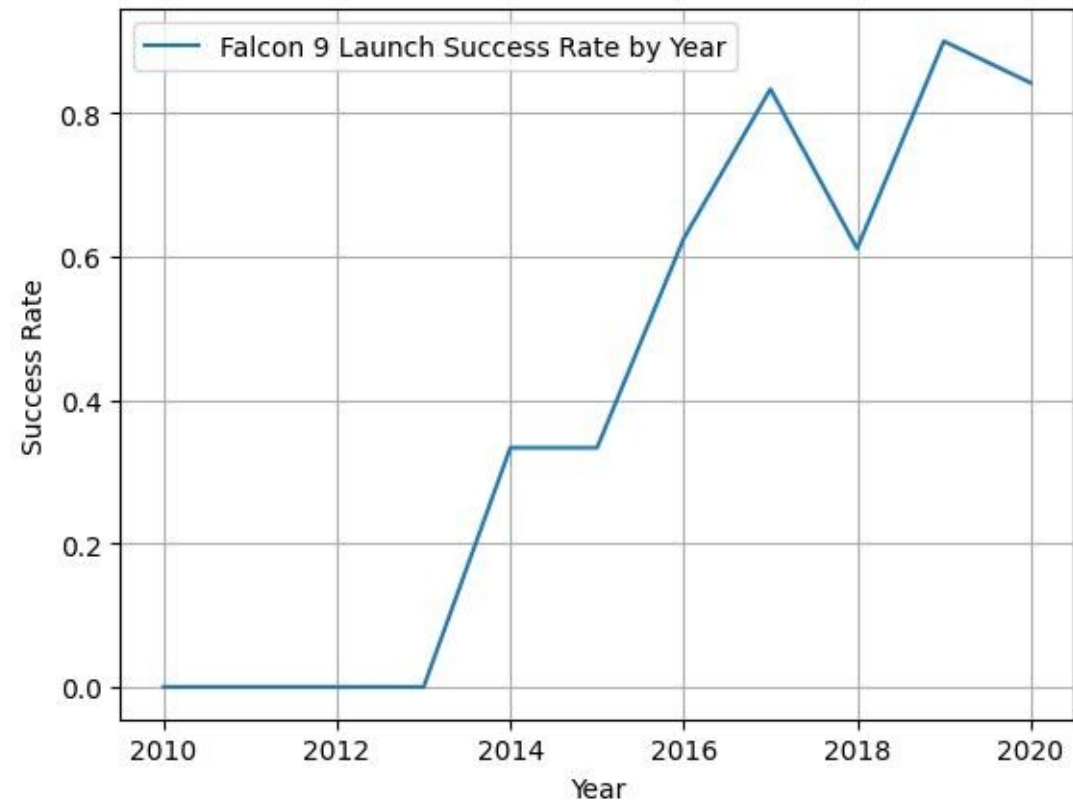


Orbit vs. Payload Mass

In the figure below, we can see that the maximum number of launches were to GTO orbit with a Payload Mass between aprox. 3000kg to 7000kg. In the 2nd place we've got ISS with a Payload Mass ranging between less than 2000kg to a little above 3500kg with outliers.



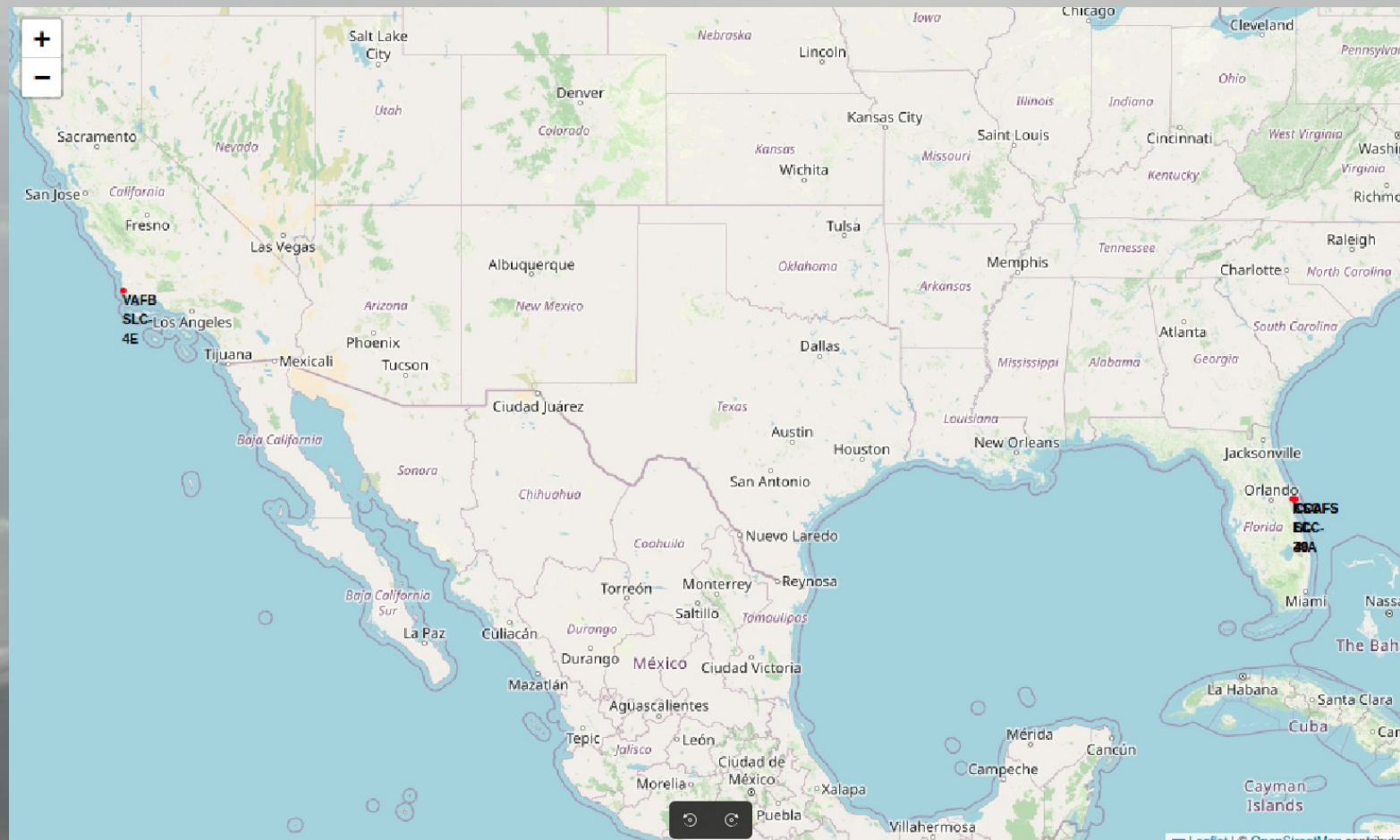
Success Rate vs. Year



Folium Launch Sites Maps

Two red dots indicating the location of the USA's Spacial Launch Facilities.

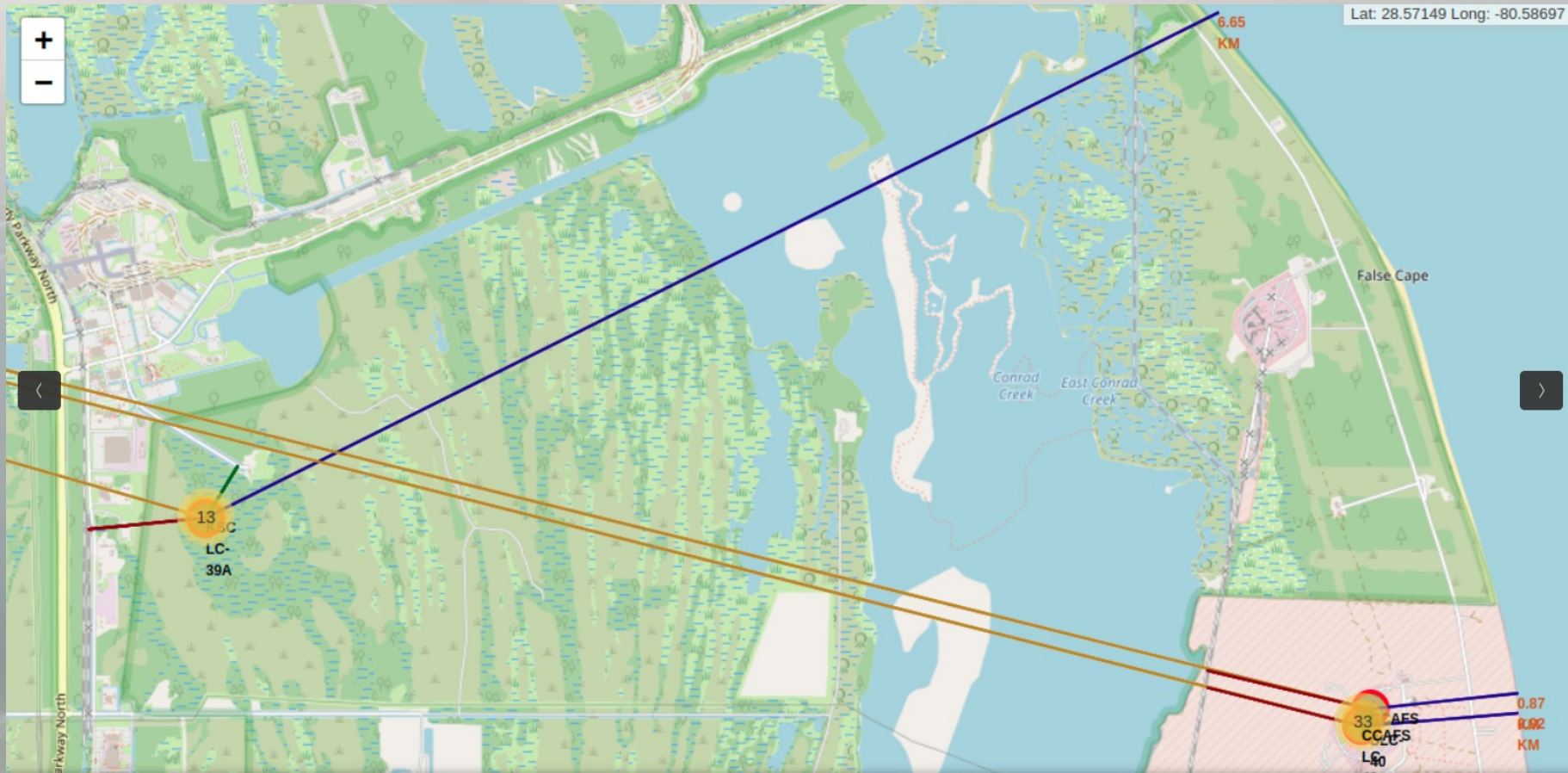
At the extreme west close to the USA – Mexico frontier we have the Vandenberg (VAFB SLC – 4E). At the extreme east we have Kennedy Space Center (KSC-LC 39A), Cape Canaveral (CCAFS SLC-40), Cape Canaveral (CCAFS LC-40).



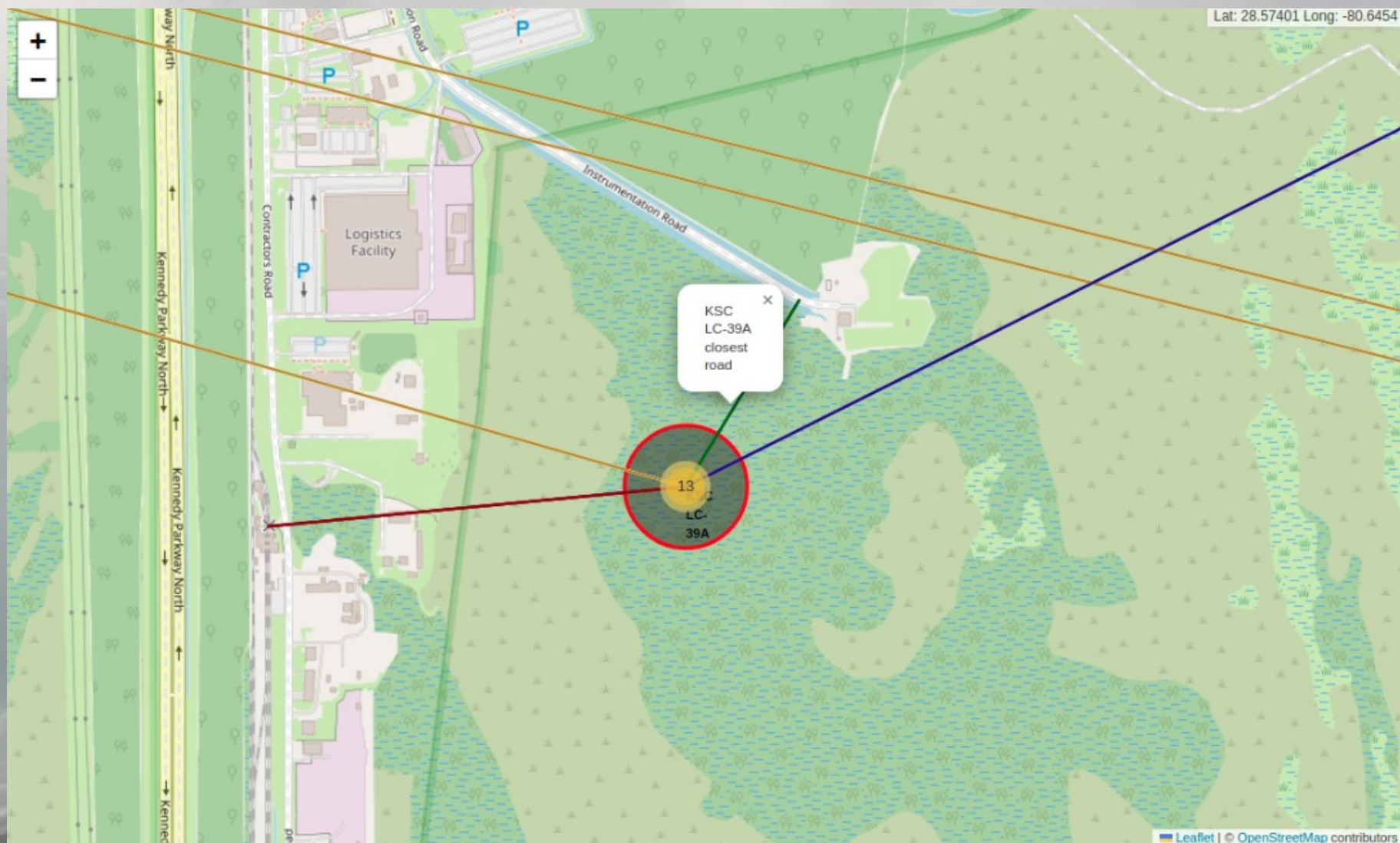
In the picture below we can clearly see KSC LC-39A, CCAFS SLC-40 & CCAFS SLC-40 sites,



Below, KSC LC-39A, CCAFS SLC-40, and CCAFS SLC-40 Closest city, coastline, railway, road, etc. are showing,



Here we can see kSC LC-39A Closest Road is Instrumentation Road,



Plotly Dash Interactive Charts

We will build a Plotly Dash application for users to perform interactive visual analytics on SpaceX launch data in real-time.

This dashboard application contains input components such as a dropdown list and a range slider to interact with a pie chart and a scatter point chart.

Adding a Launch Site Drop-down Input Component,

```
html.Div(["site-dropdown: ",
        dcc.Dropdown(id='site-dropdown',
                     options=[{'label': 'All', 'value': 'ALL'},
                              {'label': 'CCAFS LC-40', 'value': 'CCAFS LC-40'},
                              {'label': 'CCAFS SLC-40', 'value': 'CCAFS SLC-40'},
                              {'label': 'KSC LC-39A', 'value': 'KSC LC-39A'},
                              {'label': 'VAFB SLC-4E', 'value': 'VAFB SLC-4E'}],
                     value='ALL',
                     # placeholder="Coose a Launch Site",
                     searchable=True),
        html.Div([html.Div(dcc.Graph(id='success-site_pie-chart'))],
                 style={'display': 'flex'})],)
```


Success-pie-chart site dropdown callback function,

```
@app.callback(Output(component_id='success-site_pie-chart', component_property='figure'),
               Input(component_id='site-dropdown', component_property='value'))

def charts(entered_site):

    if entered_site == 'ALL':
        fig_1 = px.pie(spx_site_success, 'Launch Site',
                       'Success Launches',
                       title='Number of Success Launches per Site')
    else:
        spx_entered_site = spacex_df[spacex_df['Launch Site']==entered_site]
        spx_entered_site = spx_entered_site.pivot_table(index='class', values='Launch Site',
                                                         aggfunc = 'count').reset_index()

        fig_1 = px.pie(spx_entered_site,
                       'class',
                       'Launch Site',
                       title = 'Success (1) vs. Failed (0) Launches from ' + entered_site + ' site')

    return fig_1
```


Adding a Range Slider to Select Payload,

```
html.Div(["payload-slider: ",
        dcc.RangeSlider(id='payload-slider',
                        min=Min_Pld,
                        max=Max_Pld,
                        value=[Min_Pld, Max_Pld]
                        ),
        #update mode='drag',
        html.Div([html.Div(dcc.Graph(id='success-payload_scatter-chart'))],
                style={'display': 'flex'})],))
```

Success-payload scatter-chart callback function to render the scatter plot,

```
@app.callback(Output('success-payload_scatter-chart', 'figure'),
              Input('payload-slider', 'value'))

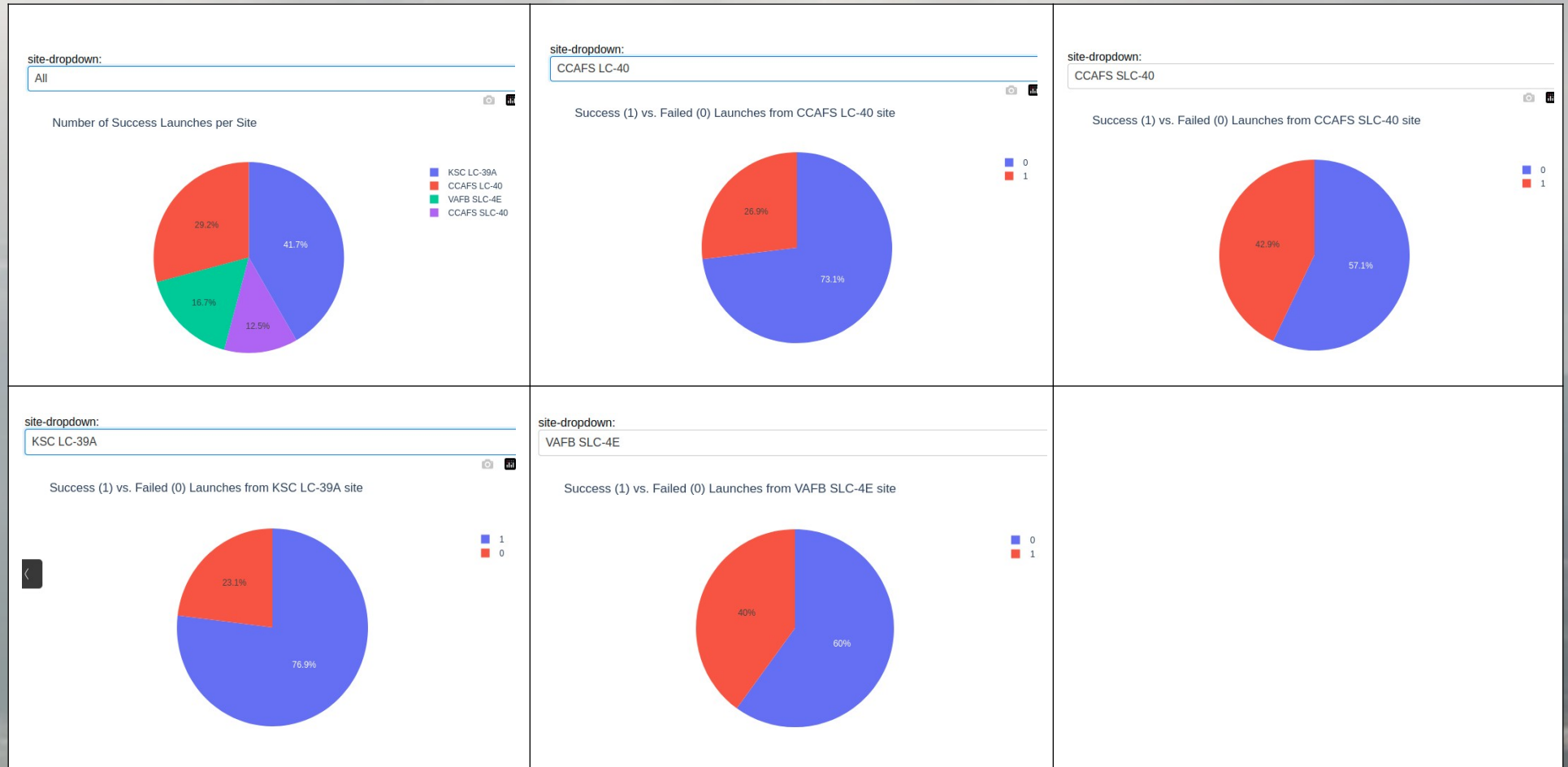
def get_scatter_plot(entered_payload):

    Min = entered_payload[0]
    Max = entered_payload[1]
    spx_entered_payload = spacex_df[(spacex_df['Payload Mass (kg)'] >= Min) & (spacex_df['Payload Mass (kg)'] <= Max)]

    fig_2 = px.scatter(spx_entered_payload, 'Payload Mass (kg)', 'class')

    return fig_2
```

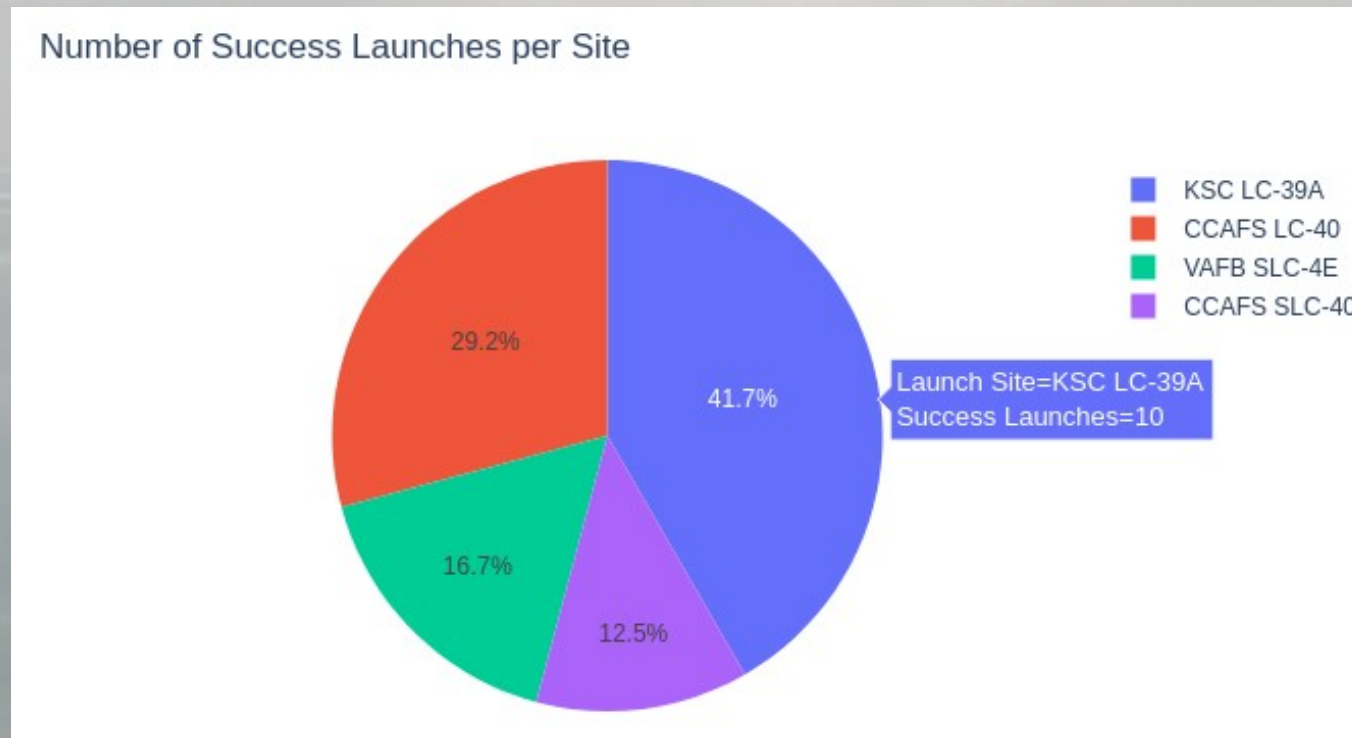
Different sites selections pie charts,



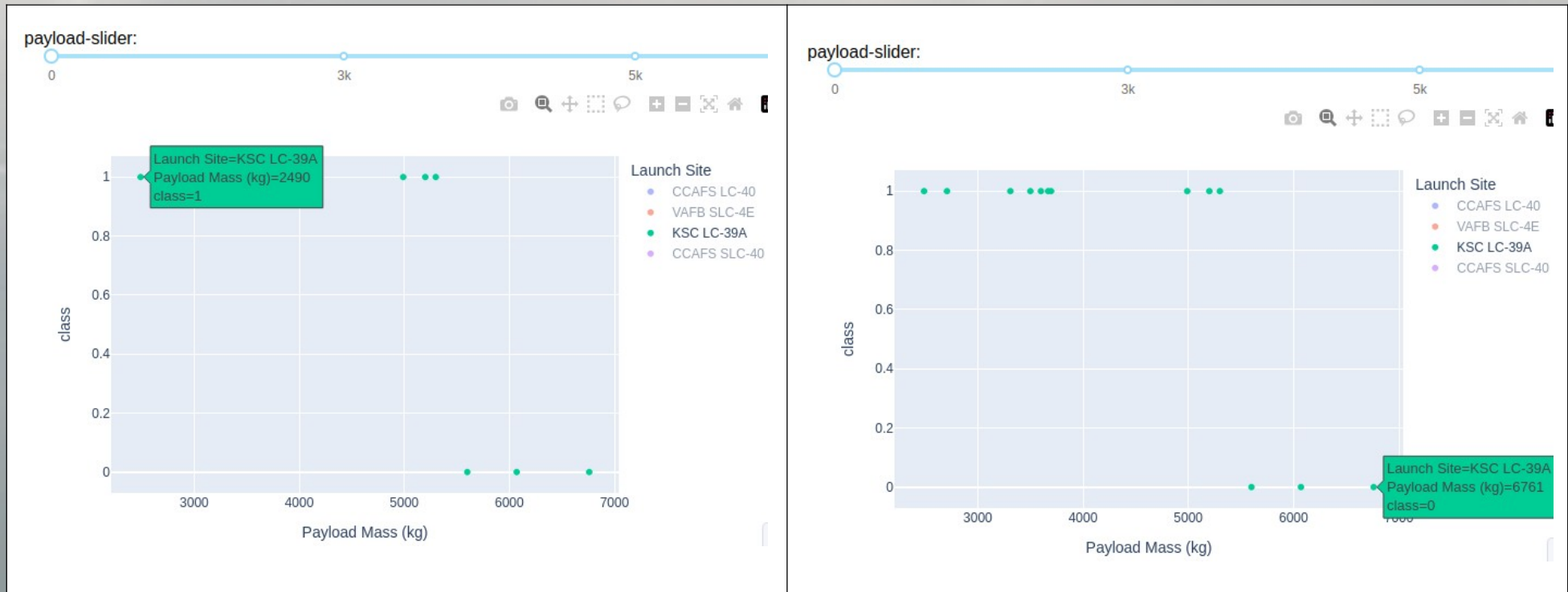
Different slider settings scatter charts,



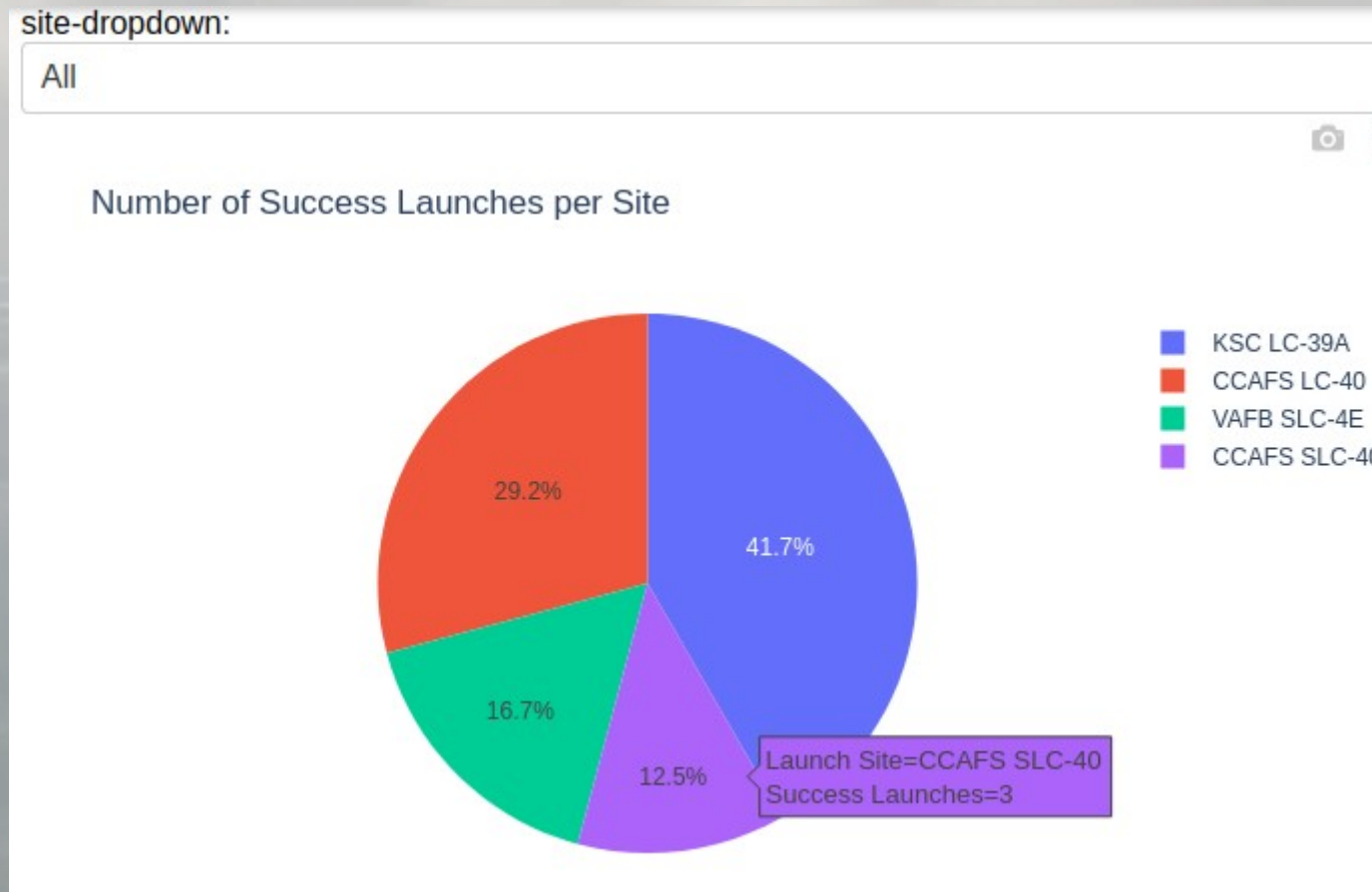
The site with largest successful launches & the highest launch success rate is KSC LC-39A as you can see in the chart below,



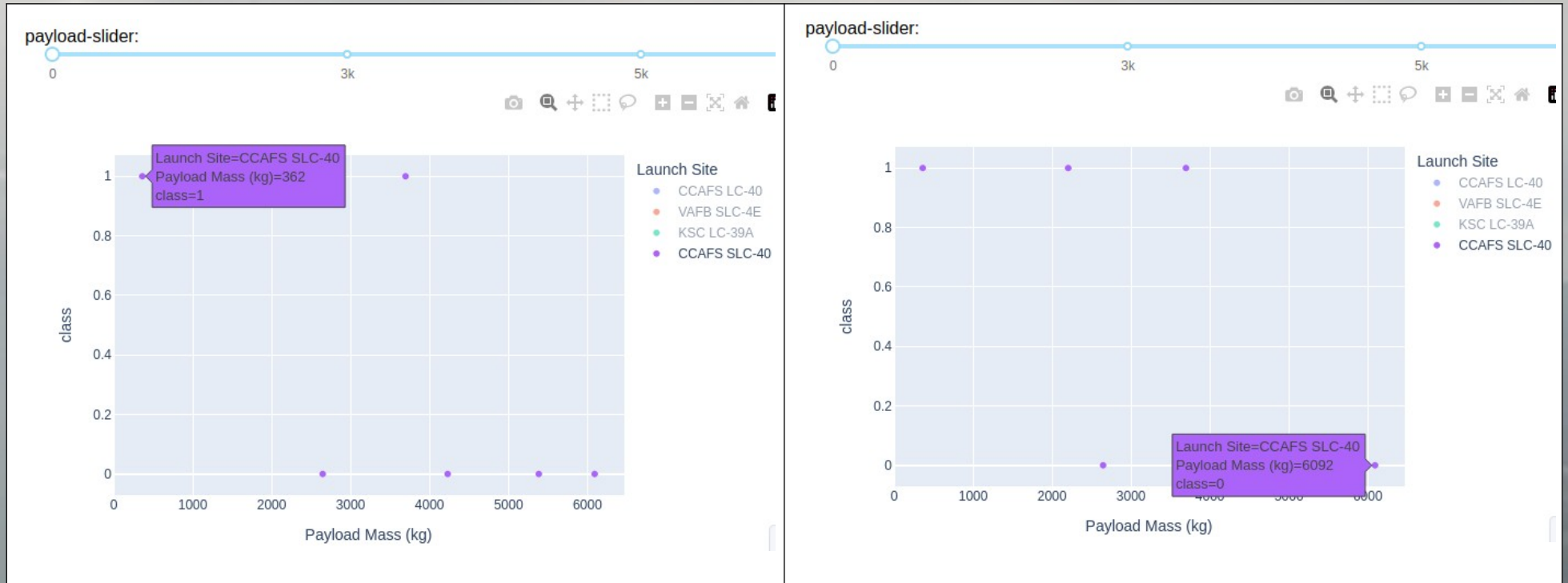
The payload range with the highest launch success rate is 2940Kg - 6761Kg corresponding to KSC LC-39A, as you can see in the below figures,



The site with the lowest launch success rate is CCAFS SLC-40,



The payload range with the lowest launch success rate is 362Kg - 6092Kg corresponding to CCAFS SLC-40 site,



F9 highest launch success rate booster version (v1.0, v1.1, FT, B4, B5, etc.),

```
spx_max_success_booster = spacex_df[(spacex_df['Launch Site']=='KSC LC-39A')]  
spx_max_success_booster['Booster Version Category'].unique()  
  
array(['FT', 'B4', 'B5'], dtype=object)
```

Predictive models (Building, Evaluating, Refining)

In this Data Science project, the problem to resolve is a **Binary Classification problem**.

We use the previously built data to predict whether SpaceX will attempt to land a rocket or not.

We will train four different classification models using **Cross Validation** to find the best **Hyper Parameters** via **Grid Search** for each model. The selected models to train are,

- a Logistic Regression Model,
- a Support Vector Machine Model,
- a Classification Tree Model, and
- a K_Nearest_Neighbors Model (KNN) .

The steps to achieve our goals are,

- Create a NumPy array from the column Class in data and assign it to the variable Y.
- Standardize the data in X then reassign it to the variable X using the transform provided.
- Split the data into training and testing data.
- Divide our training data into validation data, a second set used for training data.
- Train the models selecting hyperparameters using Grid Search with Cross Validation.

Logistic Regression Model

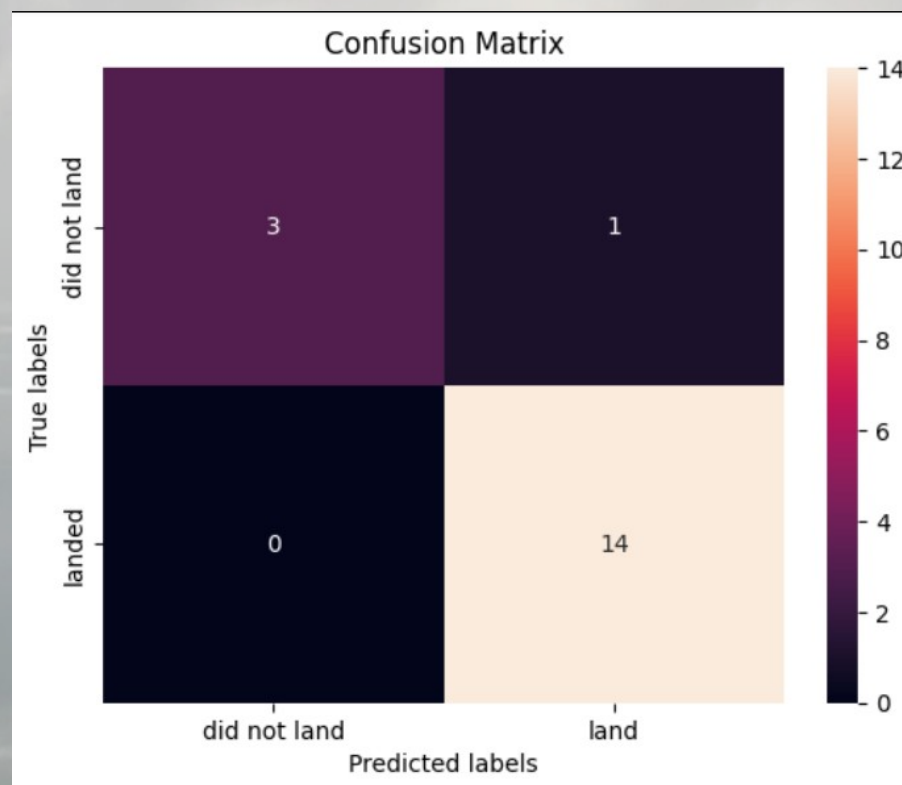
In the picture below we can see the definition of the GridSearchCV for our Logistic Regression Model,

```
GridSearchCV
GridSearchCV(cv=10, estimator=LogisticRegression(max_iter=1000),
             param_grid={'C': [0.01, 0.1, 1], 'penalty': ['l2'],
                         'solver': ['lbfgs']})
best_estimator_: LogisticRegression
LogisticRegression(C=0.1, max_iter=1000)
```

The best parameters automatically selected are:

- 'C': 0.1, 'penalty' = 'l2',
- 'solver': 'lbfgs'

Accuracy in Training Set = 0.8035714285714285



Metrics on test set:

logreg_cv accuracy: 0.9444444444

logreg_cv F1 score: 0.9655172414

Support Vectors Machine Model (SVM)

In the picture below we can see the definition of the GridSearchCV for our Support Vectors Machine Model,

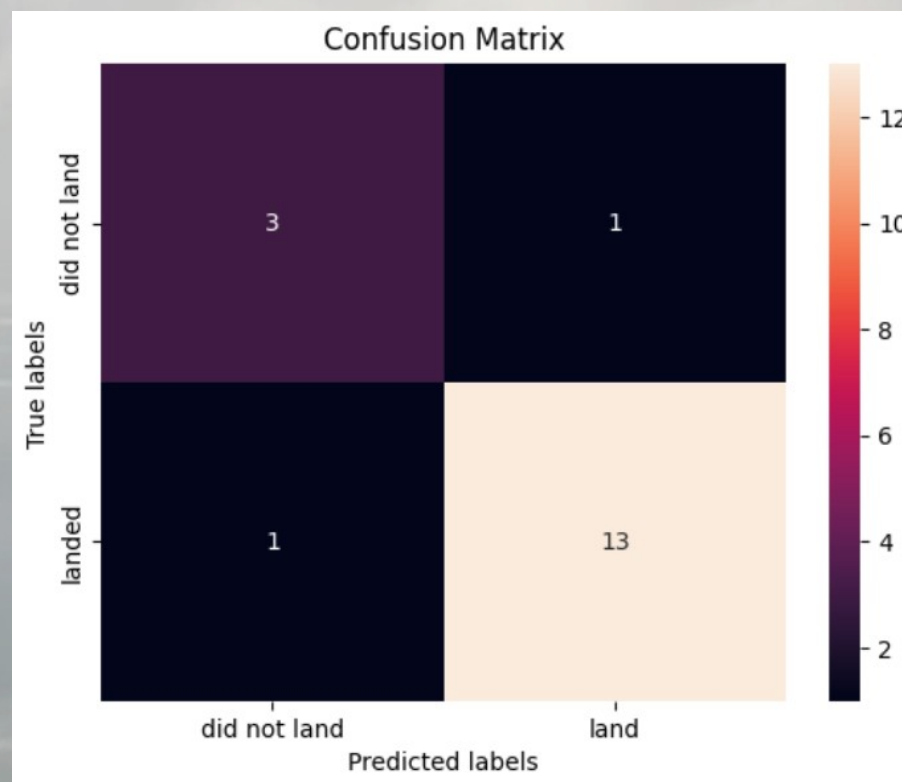
```
GridSearchCV(cv=10, estimator=SVC(max_iter=1000),
             param_grid={'C': array([1.00000000e-03, 3.16227766e-02, 1.00000000e+00, 3.16227766e+01,
                                     1.00000000e+03]),
                         'gamma': array([1.00000000e-03, 3.16227766e-02, 1.00000000e+00, 3.16227766e+01,
                                         1.00000000e+03]),
                         'kernel': ('linear', 'poly', 'rbf', 'sigmoid')})

best_estimator_: SVC
SVC(C=np.float64(1.0), gamma=np.float64(0.03162277660168379), kernel='sigmoid',
    max_iter=1000)
```

The best parameters automatically selected are:

- 'C' = np.float64(1.0)
- 'gamma' = np.float64(0.03162277660168379)
- 'kernel' = 'sigmoid'

Accuracy in Training Set = 0.832142857142857



Metrics on test set:

svm_cv accuracy: 0.8888888889

svm_cv F1 score: 0.9285714286

Tree Classifier Model

In the picture below we can see the definition of the GridSearchCV for our Tree Classifier Model,

```
GridSearchCV
GridSearchCV(cv=10, estimator=DecisionTreeClassifier(),
             param_grid={'criterion': ['gini', 'entropy'],
                          'max_depth': [2, 4, 6, 8, 10, 12, 14, 16, 18],
                          'max_features': ['auto', 'sqrt'],
                          'min_samples_leaf': [1, 2, 4],
                          'min_samples_split': [2, 5, 10],
                          'splitter': ['best', 'random']})

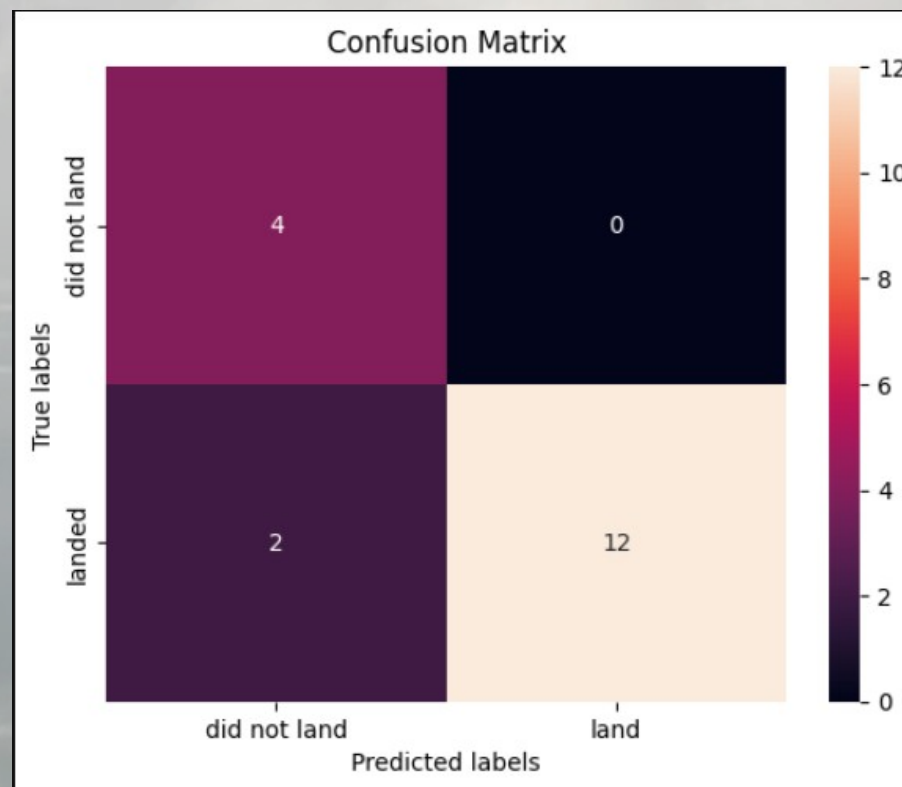
best_estimator_: DecisionTreeClassifier

DecisionTreeClassifier
DecisionTreeClassifier(max_depth=12, max_features='sqrt', min_samples_split=10)
```

The best parameters automatically selected are:

- 'criterion' = 'gini'
- 'max_depth' = 12
- 'min_samples_leaf' = 4
- 'min_samples_split' = 10
- 'splitter' = 'random'

Accuracy in Training Set = 0.8625



Metrics on test set:

tree_cv accuracy: 0.8888888889

tree_cv F1 score: 0.9285714286

K_Nearest_Neighbors Classifier (KNN)

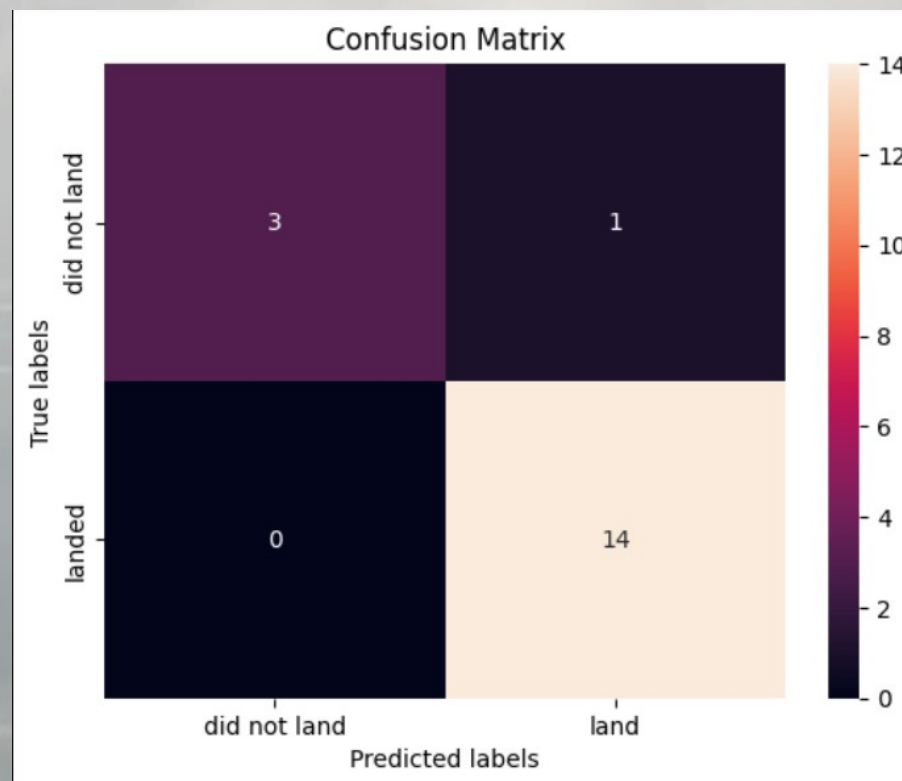
In the picture below we can see the definition of the GridSearchCV for our KNN classifier Model,

```
GridSearchCV(cv=10, estimator=KNeighborsClassifier(),  
             param_grid={'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],  
                         'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],  
                         'p': [1, 2]})  
  
▼ best_estimator_: KNeighborsClassifier  
KNeighborsClassifier(n_neighbors=6, p=1)
```

The best parameters automatically selected are:

- 'algorithm' = 'auto'
- 'n_neighbors' = 6
- 'p' = 1

Accuracy in Training Set = 0.8446428571428569



Metrics on test set:

Accuracy (knn_cv): 0.944444444444

F1 score (knn_cv): 0.9655172414

Conclusion

Throughout all stages of this data science project, we obtained valuable information to learn about and understand the company we hypothetically want to compete with, and we learned about the aspects that define it, such as the variety of services they provide, their clients, the most in-demand services, their reliability, their strategies and technologies, their vision, etc.

For the predictions, we defined, trained, and evaluated four different binary classification models: a Logistic Regression, a Support Vectors Machine, a Classification Tree, and a K_Nearest Neighbors Classifier. Their results can be seen in the table below,

logreg_cv	Training Set: Accuracy = 0.8035714285714285	tree_cv	Training Set Accuracy = 0.8625
	Test set: Accuracy = 0.94444444444 F1 score = 0.9655172414		Test set: Accuracy: 0.88888888889 F1 score: 0.9285714286
svm_cv	Training Set Accuracy = 0.832142857142857	knn_cv	Training Set Accuracy = 0.832142857142857
	Test set: Accuracy = 0.88888888889 F1 score = 0.9285714286		Test set: Accuracy: 0.94444444444 F1 score: 0.9655172414

Finally, according to their metrics in the table above, we can see that the two best classifiers are the Logistic Regression and KNN models, while among them the winner is our KNN model, because although both coincide in their scores on the test set they differ on the training set, with the KNN model having a higher accuracy score.

...

Repository Links:

<https://github.com/Leus74/IBM-Data-Science-Specialization-Capstone-Project.git>